

Ускорение инференса нейронных сетей: сравнительный анализ методов оптимизации

Промежуточный отчёт

Авторы:

Фоменко Андрей Алексеевич
Старощук Богдан Павлович
Акимов Тимур Арсенович

Цель проекта

Реализовать и сравнить несколько pipeline ускорения
инференса UNet-модели на задаче сегментации изображений:
PyTorch FP16 → torch.compile → TVM → PTQ/QAT → Pruning

0 Contents

1	Введение	2
1.1	Постановка задачи	2
1.2	Исследуемые методы ускорения	2
2	Архитектура и инфраструктура	2
2.1	Модель: UNet	2
2.2	Функция потерь и метрики качества	2
3	Текущее состояние: smoke test	3
3.1	Цель smoke test	3
3.2	Результаты обучения (micro test)	3
3.3	Бенчмарк инференса: baseline vs torch.compile	3
3.4	Интуиция и выводы по эксперименту	4
4	Дальнейший план работы	4
4.1	Ожидаемые результаты	4
5	Промежуточные выводы	4

1 Введение

Наш отчёт описывает промежуточные результаты работы над проектом по ускорению инференса нейронных сетей. Ключевая задача это получение воспроизводимого бейстайла и последовательного применения ряда техник оптимизации, с фиксацией изменения скорости и качества.

1.1 Постановка задачи

В качестве целевой задачи выбрана **бинарная сегментация изображений** на датасете *Oxford-IIT*. Датасет содержит 7 349 изображений кошек и собак с пиксельными масками (trimap). Мы бинаризуем маску: класс «питомец» *vs.* фон.

Выбор этого датасета обусловлен:

- небольшим размером (≈ 800 МБ), что сильно ускоряет эксперименты;
- наличием готовой загрузки через `torchvision`;
- достаточной сложностью для получения нетривиального бейстайла.

1.2 Исследуемые методы ускорения

1. **Baseline FP16** - стартовая модель;
2. **torch.compile** - JIT-компиляция вычислительного графа (`inductor`);
3. **TVM (Apache Relax/TIR)** - альтернативный компилятор, генерирует оптимизированный код под целевое железо;
4. **PTQ** (Post-Training Quantization) - статическая квантизация int8 без переобучения;
5. **QAT** (Quantization-Aware Training) квантизация с дообучением;
6. **Pruning / 2:4 Sparsity** спарсифицируем веса с последующим дообучением.

2 Архитектура и инфраструктура

2.1 Модель: UNet

В качестве модели используется классический **UNet** для бинарной сегментации. Архитектура состоит из симметричного энкодера и декодера со skip-connections:

- **Энкодер:** 4 блока ($\text{MaxPool2d} \rightarrow \text{Conv-BN-ReLU} \times 2$), количество каналов: $64 \rightarrow 128 \rightarrow 256 \rightarrow 512$;
- **Боттлнек:** 512 каналов;
- **Декодер:** 4 блока ($\text{Upsample} \rightarrow \text{concat} \rightarrow \text{Conv-BN-ReLU} \times 2$);
- **Выход:** 1×1 свёртка $\rightarrow$ логит бинарной маски.

Суммарное количество обучаемых параметров: **13,391,361**.

2.2 Функция потерь и метрики качества

Для обучения используется комбинированная функция:

$$\mathcal{L} = 0.5 \cdot \mathcal{L}_{\text{Dice}} + 0.5 \cdot \mathcal{L}_{\text{BCE}}, \quad (1)$$

где

$$\mathcal{L}_{\text{Dice}} = 1 - \frac{2|\hat{y} \cap y| + \varepsilon}{|\hat{y}| + |y| + \varepsilon}, \quad \varepsilon = 10^{-6}. \quad (2)$$

Метрики оценки качества:

- **Dice coefficient** (основная метрика);
- **IoU** (Intersection over Union);
- **Pixel Accuracy**.

Метрики скорости:

- latency P50/P95 при batch size = 1 (мс);
- throughput (изображений/с) при batch size = 16;
- размер модели (МБ).

3 Текущее состояние: smoke test

3.1 Цель smoke test

На первом этапе мы провели *micro test* - небольшое обучение в течение 3 эпох с целью убедиться в корректности всего пайплайна, оценить скорость сходимости и выбрать гиперпараметры для боевого запуска

3.2 Результаты обучения (micro test)

В ходе обучения наблюдается стабильное улучшение всех ключевых метрик. Значение функции потерь на обучающей выборке последовательно снижается с 0.435 до 0.317 и далее до 0.278, аналогично уменьшается и ошибка на валидации: с 0.359 до 0.269 и затем до 0.231. При этом качество сегментации уверенно растёт: коэффициент Dice увеличивается с 0.740 до 0.799 и достигает 0.829, а метрика IoU возрастает с 0.611 до 0.682 и далее до 0.722. Такая динамика свидетельствует о корректной сходимости модели и отсутствии признаков переобучения на данном этапе. В итоге можно сказать, что наблюдается стабильная сходимость без признаков деградации качества, более того Val-метрики монотонно улучшаются, что подтверждает корректность реализации

3.3 Бенчмарк инференса: baseline vs torch.compile

После получения базовой обученной модели был проведён benchmark инференса для двух конфигураций:

- **Baseline FP16** -стандартный eager execution PyTorch;
- **torch.compile** (mode = `reduce-overhead`) те компиляция графа с целью снижения overhead интерпретатора.

Table 1: Сравнение baseline и torch.compile

Метод	P50 (мс)	P95 (мс)	Throughput	Dice	IoU
Baseline FP16	13.84	20.66	47.5	0.8227	0.7157
torch.compile	13.59	13.87	84.9	0.8227	0.7157

3.4 Интуиция и выводы по эксперименту

В режиме eager execution PyTorch исполняет операции по отдельности, что приводит к частым переходам между Python и backend, множественным запускам kernel'ов и дополнительным накладным расходам на синхронизацию. Использование `torch.compile` позволяет построить единый вычислительный граф и оптимизировать его выполнение: операции объединяются (fusion), уменьшается количество kernel launch'ей и снижается overhead интерпретатора. В результате сами вычисления практически не ускоряются, что объясняет почти неизменную latency на уровне P50, однако существенно уменьшается разброс времени выполнения (P95) и значительно растёт throughput, особенно при batch inference.

Таким образом, `torch.compile` даёт заметный прирост производительности без потери качества модели, причём основной выигрыш достигается за счёт снижения накладных расходов исполнения. Эффект особенно выражен при обработке батчей, что делает данный метод важным и практически обязательным этапом базовой оптимизации перед применением более сложных техник, таких как TVM, квантизация и pruning.

4 Дальнейший план работы

4.1 Ожидаемые результаты

По завершении всех экспериментов ожидается итоговая таблица вида:

Table 2: Шаблон итоговой сравнительной таблицы (значения будут заполнены после экспериментов)

Метод	Precision	P50 (мс)	P95 (мс)	Throughput	Dice	Размер
Baseline FP16	fp16	13.84	20.66	47.5	0.8227	26.78
<code>torch.compile</code>	fp16	13.59	13.87	84.9	0.8227	26.78
TVM (Relax)	fp32	—	—	—	—	—
ONNX Runtime	fp32	—	—	—	—	—
PTQ (int8)	int8	—	—	—	—	—
QAT (int8)	int8	—	—	—	—	—
Pruning (unstructured)	fp16	—	—	—	—	—
2:4 Semi-structured	fp16	—	—	—	—	—

5 Промежуточные выводы

На текущем этапе разработан и успешно верифицирован полный pipeline обучения и оценки модели. Проведённый smoke test показал корректную сходимость уже за 3 эпохи значение метрики Dice достигло 0.829, что подтверждает правильность реализации архитектуры, функции потерь и процедуры обучения.

В ходе экспериментов была выявлена существенная проблема с производительностью MPS backend, проявляющаяся в резком замедлении обучения после первой эпохи. В связи с этим дальнейшие эксперименты планируется перенести на CUDA-GPU, что

позволит обеспечить стабильную и воспроизводимую производительность. Планируется реализовывать дальнейшие шаги на A100.

Параллельно была подготовлена инфраструктура для проведения всех ключевых экспериментов по ускорению инференса, включая `torch.compile`, ONNX Runtime, PTQ, QAT и pruning. На данный момент не реализованы только наиболее сложные компоненты а именно TVM и 2:4 semi-structured sparsity, которые будут добавлены на следующих этапах

В целом проект находится примерно на 25% от запланированного объёма. Следующим шагом является запуск полного обучения модели на GPU с сохранением чекпоинтов, после чего будет проведён систематический бенчмарк всех методов ускорения с последующим сравнительным анализом их влияния на скорость и качество.